

Getting Started with the Pacman Game Environment

PREREQUISITES: At this stage, you should have (i) a GitHub account; (ii) an account on the evaluation server; and (iii) a copy of the Pacman game environment (code repository) on your local machine. If you don't satisfy these prerequisites, refer to the Assignment Installation Guide.

In this document, we describe the functioning of the Pacman game environment and the organisation of its source code and highlight various functions and utilities which you may find useful in your implementation. We'll cover

- The fundamental game loop and game rules (incl. termination conditions)
- The **GameState** object and its main functions/features
- The organisation of the code
- The purpose and organisation of the layouts

1. The Game Loop

When **pacman.py** is run, the initial game configuration is loaded from the specified map layout (**.lay** file). Controllers are then initialised, and any agents mentioned in the layout file are added to the map. The core gameplay loop then begins.

The game is divided up into iterations, where each iteration begins with the game environment asking for an action from the Pac-Man controller. The chosen action is then applied, resulting in an update to the **GameState** object (more on this in the next section). If there is a ghost on the map, their controllers are queried next, again resulting in updates to the **GameState**. The process continues in a loop as follows:

GameState → [Pacman action] → GameState → [Ghost action] → GameState → [Pacman action] → ...

Each agent is identified with an ID, and the turn order is always the same in each loop. Pacman is also always the first agent to move at the beginning of the game.

Certain actions from Pacman are rewarded with points from the game environment. The point scoring (which is additive) works as follows:

- Move to an adjacent tile: -1
- Eat a food dot: +10
- Eat a power pellet (capsule): +0
- Eat a "scared" ghost: +200 – If Pacman contacts a "non scared" ghost, the game ends with a score of 0
- Eat the last food dot: +500

The ghost temporarily enters a "scared" state when Pacman eats a power pellet (capsule) — they look like bigger versions of food dots, and their "scare" effect expires after 40 moves.

The game ends when one of the following termination conditions is satisfied:

- Pacman has eaten all the food dots on the board
- Pacman is caught by the ghost.
- Time has run out.

2. The GameState class

The current state of the game (configuration of the maze, positions of the agents, locations of remaining food and the current score) is stored in a **GameState** object.

This is an important class that you should become familiar with. Its implementation can be found in the file **pacman.py**. When an agent is asked to select an action, they will have access to the current state of the game via an instance of this class. Some potentially useful methods/attributes of the **GameState** class are the following:

- **getLegalActions** - Given agent index *i*, returns a list of the legal actions.
- **generateSuccessor** - Takes an agent index *i* and action and creates a new **GameState** object that would result from agent *i* taking the given action.
- **getPacmanPosition** - Return an (x,y) tuple with the location of Pacman on the grid
- **getGhostStates** - Returns a list of **AgentState** (defined in **game.py**) objects that store information about the ghost state. Primarily useful for checking how long the ghost will be scared for when Pacman has eaten a capsule.
- **getScore** - Returns Pacman's current score
- **getCapsules** - Returns a list of (x,y) positions of the capsules on the grid
- **getNumFood** - Returns the number of food dots remaining on the board.
- **getFood** - returns a 2D array where the value at array position (x, y) is true if there exists a food dot at the corresponding location on the grid.
- **getWalls** - returns a 2D array where the value at array position (x, y) is true if the corresponding position is a wall (i.e., not traversable).
- **hasWall** - Given an x,y location, returns **True/False** if that location is a wall.
- **hasFood** - takes an x,y location as input and returns **True** if that location has food and **False** otherwise.
- **isWin** - **True** if the current game state is a win for Pacman and **False** otherwise.
- **isLose** - **True** if the current game state is a loss for Pacman and **False** otherwise.

There are more attributes and methods that may be useful, and you should read the code for the **GameState** class to get the specific details of the methods above. What is listed above is just to get you started. If you are beyond "getting started", then it would also be worth reading the **AgentState** class in **game.py**.

3. Agent Controllers

The behaviour of every agent in the Pacman game is described by a corresponding controller. All controllers inherit from a base class, **Agent** (defined in **game.py**), which defines a single method, **getAction**, that must be implemented.

The simulator interacts with each of the agents in the game via the **getAction** method. This method takes a **GameState** parameter (see above) and returns a next action for the agent to

take, specified via the **Directions** class (defined in **game.py**). To see an example agent, take a look at the implementation in **GoWestAgent.py**. The different controllers you must develop in this unit will all implement the **getAction** method differently.

Most of the agents you create will implement **getAction** in a way that is reactive to the current game state; that is, they receive a **GameState** object, perform some computations and then return an action.

In Part 1 of the assignment, we will work with a specialised type of agent called **SearchAgent**, whose implementation can be found in the file **SearchAgents.py**. These agents are initialised with a search problem to solve (you'll formulate the problems) and a search algorithm to solve that problem (you'll implement the algorithms).

When a **SearchAgent** solves a problem, the resulting plan, a sequence of actions, is saved. When the **getAction** method of this agent is called, the next action in the plan is returned and executed. The problem is considered solved when the agent has executed all the saved actions.

You should familiarise yourself with the **SearchAgent** class. The problem and solver are given in the **__init__** method. The plan is computed and saved in the **registerInitialState** method. The actions are read from the plan in the **getAction** method.

4. Helpful Utilities

The file **util.py** provides a number of helpful data structures. You may find it helpful to refer to these implementations when developing your controller. They include:

- Stack
- Queue
- Priority Queue

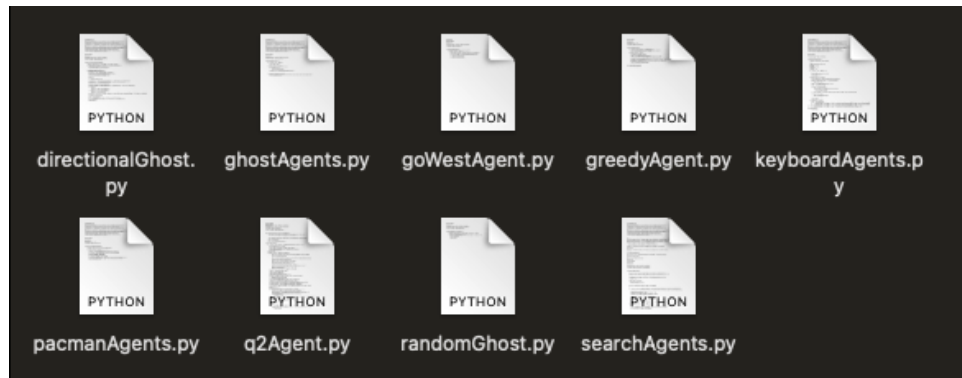
5. How the code is organised

In this section, we describe the organisation of the pacman project files. You will need to navigate, read and understand different parts of the project in order to develop your controller. The code is organised into the following directories:

- [project root]
 - |- agents
 - |- layouts
 - |- logs
 - |- problems
 - |- solvers

5.1 Agents

The 'agents' directory contains code for the different agents that participate in the game, so Pacman, of course, and the ghost. The contents of the directory should look like this.



Important files:

- **SearchAgents.py**: This file contains the base class implementation for agents that you will develop in Part 1 of the assignment. A description of how a **SearchAgent** agent works is given in Section 3. **You will not need to modify this file, but you should be familiar with how the implementation works.**
- **q2Agent.py**: You will modify this file in Part 2 of the assignment. You will implement *Alpha-Beta* search in the **getAction** method of this agent.

5.2 Layouts

The layouts directory contains maps that you can use to test your implementations for each question. **These are not the same maps that we evaluate you on, but they are similar.** The prefix of the filename should reflect the question it should be used in. smallClassic.lay and testMaze.lay are also included as part of the demo for this document. An example of the contents of a layout file is shown below.

```
%%%%%%%%%
%. . . . .%G %. . . .%
%. % . . . % % % . . . %
%. % O . % . . . . . % . O % . %
%. % . % . % % % % . % . % . %
%. . . . . P . . . . . %
%%%%%%%%%
```

Explanation of symbols

- % = Wall (non-traversable location)
- P = The starting location of Pacman
- G = The starting location of the ghost
- . = A food dot
- o = A power pellet/capsule

5.3 Logs

The `logs` directory stores the log files that are created each time you run `pacman.py` with the `-o` flag. More information about the logs can be found in Section 8.

5.4 Problems

This directory contains a Python file for each of the tasks in Part 1 of the assignment:

- `q1a_problem.py`:
- `q1b_problem.py`:

The classes in these files have the same three methods:

- `getStartState`
- `isGoalState`
- `getSuccessors`

You will implement each class to describe the problem representation for each of the questions in Part 1 of the assignment. More information about how to approach this task (and Part 2) can be found in Section 6.

5.5 Solvers

There are two files in this directory. Each one contains a function stub where you will implement the corresponding search strategy for each of the questions in Part 1 of the assignment.

- `q1a_solver.py`, which contains the `q1a_solver` function stub.
- `q1b_solver.py`, which contains the `q1b_solver` function stub.

6. Implementing your controller

In Sections 6.1 and 6.2, we give an overview of how you might approach Q1(a). This information is also useful as a general guide for Q1(b). Then, in Section 6.3, we give an overview of how you might approach Q2.

6.1 Model the problem (Assignment Part 1)

The first thing to do is model the problem concretely, which you will achieve by modifying the classes `q1a_problem` and `q1b_problem`. Constructed by the game environment, this class stores the initial state of the game board (a `GameState` object) and a series of function stubs, which you will implement. The stubs are as follows:

- **`getStartState`**: this function takes no parameters and returns the **initial state** of the problem (this is a data structure analogous to **but not necessarily the same** as `GameState`); i.e., it represents the configuration of the map at the start of the game.
- **`isGoalState`**: Given a **current** state, this function returns true if that state satisfies the goal conditions of the problem (i.e., you get to decide what those are).
- **`getSuccessors`**: Given a **current** state, this function returns a set of states, each reachable from the current state by applying a valid action having an associated cost.

As part of this task, you will get to decide which features of the environment (`GameState`) need to be represented in your corresponding search state. **You get to decide what the search state data structure looks like**, and which attributes it contains (e.g., the position of Pacman determined by your algorithm might be a helpful thing to store)

6.2 Implement a search algorithm (Assignment Part 1)

With a model of the problem in hand, you must turn your attention to the next issue: how to compute a solution. Here you are asked to implement the function stub `q1a_solver`, which is found in the `solvers` directory.

The function `q1a_solver` is called by the search agent when it is initialised. It receives an instance of the problem model (that you wrote earlier) as a parameter. Its job is to return a sequence of actions which moves Pacman from start state to goal state. Implement this stub by undertaking a state-space search. Refer to the functions in your problem model to initialise the search (`getStartState`), to advance the search (`getSuccessors`) and to test for a solution (`isGoalState`). **Look for the key data structures in `util.py` to make implementing your solution easier.**

The expected output from the search function is a list of actions denoted using the `Directions` class in `game.py` that fully specifies the sequence of steps for Pacman. When the search returns this sequence of steps, the simulator executes every action of the sequence, and scores the performance of Pacman. The game will end

- if all the actions in the sequence have been executed, even if there are food dots remaining on the map
- when Pacman has collected all the food dots
- when time runs out

In Q1(a), Pacman must collect a single food dot. However, in Q1(b), there are several food dots to collect, but your algorithm may decide that it is not worth it to collect every food dot.

Make sure you solve the problem completely before returning from this function!

6.3 How to Approach Assignment Part 2

For this part of the assignment, we will use the full `GameState` data structure from the Pacman environment -- instead of writing our own custom state-space representation as in Part 1. The `GameState` data structure contains a range of (now useful) features that you may want to consider when developing your controller (see Section 2).

For the same reason, we also no longer need to specify a problem model. Instead, we will use the (implicit) model provided by the game environment. In other words:

- The game environment determines the initial state (i.e., we no longer need to write `getStartState`)
- The game environment determines when the game is won or lost (i.e., we no longer need to implement `isGoalState`)
- The game environment provides an implementation of `getSuccessors` (i.e., we no longer need to implement that, either)

You need to implement your adversarial search in the **getAction** method of the **Q2_Agent** class. This method takes a **GameState** object representing the current state of the game and should return an action for Pacman. You will encounter a ghost in this part, and your adversarial search should be designed to consider the possible actions of this ghost. Review Section 2 for methods of **GameState** that you can use to interact with the ghost in the game.

Your core algorithm must be alpha-beta search, but you are free to implement any optimisations you like. There's no restriction on the depth of the search. Keep in mind that each game on the server has a **30-second time limit**. As the server configuration may differ from your local environment, the execution time and resulting score may not exactly match what you observe when running the game locally. Consequently, performing a deeper search is not always advantageous, as the additional computation may cause your agent to exceed the time limit or reduce its overall performance.

7. Testing and Debugging

This section discusses how to evaluate your agents and some general strategies to help you when your agents do not behave as you intended.

7.1 Evaluator.py

We've included a Python script, `evaluator.py`, that you can use to produce a summary of your performance across all parts of both questions on the sample layouts. To run this script, you will need to install **pandas**, **tqdm**, and **tabulate**. With your conda environment active, run the following:

```
> conda install pandas
> conda install tqdm
> conda install tabulate
```

Navigate to the directory with the **evaluator.py** script, then run the following.

```
> python evaluator.py
```

Confirm whether or not you have the latest version of the environment code (if you don't stay up-to-date, then you may not be evaluated correctly). The evaluator will attempt to run your implementation for each question on all the sample layouts provided and produce a table summarising your results. Obviously, if you have not attempted a question, then the evaluator will fail on that question. If you don't want the evaluator to run the layouts for a particular question, then you can add the following:

--q1a

Tells the evaluator not to run on the layouts for q1a

--q1b

Tells the evaluator not to run on the layouts for q1b

--q2

Tells the evaluator not to run on the layouts for q2

For example, the following will only run q2:

```
> python evaluator.py --q1a --q1b
```

The scores you will see are not the same as your scores on the evaluation server because these layouts are different. This script has been included to let you see a holistic view of your performance and to aid you in performing experiments.

7.2 Debugging

This section contains some general tips and strategies for how to test and debug your agent controllers. These strategies will help you to identify how and when a problem occurs, and they may be helpful to form a hypothesis about what types of problems could be occurring in your implementation. Try these strategies **before you ask for help**.

- **Change parameters** – You might find your agent is performing well on some layouts and not others. Try a different layout and see where your agent stops performing the way you expect it to. You might find there is a specific piece of the layout that is causing your agent trouble. Or it may be that the inclusion of capsules or ghosts is problematic. Watch your agent in a variety of scenarios and you will likely isolate the problem.
- **Inspect the log files** – The log files should contain timestamps with some function calls, observations that your agent receives, actions that your agent takes, states, and other metrics. Look for anything unexpected. Combine this with the first tip to connect what you're observing in Pacman's behaviour with the way Pacman is interacting with the simulator based on your code.
- **Run multiple games** – You can add -n as a command line argument to run multiple games on the same layout (see Section 9 for details). This is likely to be useful in part 2 of the assignment, as the ghost's moves are random.
- **Use a text output** – You can view the game as a textual output by including '-t' as a command line argument (see Section 9 for details). Unlike the graphical output, the textual output allows you to view the entire history of the game, and it doesn't disappear when the game finishes.
- **Master the small layouts before tackling the big ones** – There are some very small layouts and some very large layouts. If your agent hasn't mastered the small ones, then it is unlikely that testing them in the large layouts will help. If your agent is performing very well but there are some specific edge-case-type scenarios where it doesn't perform as expected, then it is easier to isolate these scenarios on a smaller map than a larger one.
- **Use the -c flag** – Without this flag, the game environment will not catch and process timeouts and exceptions
- **Use print statements in your code** – A single well-placed print statement can be all it takes to debug your code. They're easy to use and incredibly helpful.
- **Use the [python debugger](#)** – a little more work than print statements, but it may be more helpful.

What if none of these steps works?

Check Ed to see if someone else has had a similar problem to you. **You can't post specifics about your solution to the assignment**, but you may find something useful there. If there is a problem with our code and not yours, then chances are someone else has already encountered it. If not, then you can make a post and maybe help everyone out.

Important: Do **not** share your assignment code on Ed when asking for help. Likewise, you must **not** share your code with other students to explain or help them resolve their issues. If you need assistance, describe the problem, error messages, or expected behaviour without revealing your solution.

Where else can I get help?

The best thing is to come to consultation sessions.

When should you seek help?

As early as possible! The teaching team cannot help you directly implement your controller, but they can help you improve your understanding of search algorithms, and they may be able to point you in the right direction if you are having trouble understanding the operation of the game environment.

What if the bug is in the game code provided?

If you suspect that a bug is not in your code, then you should check (and post) on Ed. It is a good idea to check the forum posts regularly even if things are going well. Check out Section 10 for details on how to report a suspected issue in our code.

8. Log files

When you run the code with the `-o` flag, the logger is instantiated and logs function names, layout, number of agents, observation in each loop, action taken in each loop, state after action is taken, move history of Pacman, and other metrics with timestamps for you.

Log files are saved in the `logs` directory. You can use the log files to check the state of the PacMan board during the game, the actions PacMan has taken, and the final results. All metrics logged are named. If you are unsure what exactly is logged, look up what function name was logged before the metric and have a look into the code of that function.

9. Command Line Parameters

You can instantiate the `pacman.py` script with different parameters, which affect the type of game that will be played. You should become familiar with these options, as they will be helpful during your development and testing.

Usage:

USAGE: python pacman.py <options>

EXAMPLES:

(1) python pacman.py

 - starts an interactive game

(2) python pacman.py --layout smallClassic --zoom 2

```
OR python pacman.py -l smallClassic -z 2
    - starts an interactive game on a smaller board, zoomed in
```

Options:

```
-h, --help                show this help message and exit
-n GAMES, --numGames=GAMES
                           the number of GAMES to play [Default: 1]
-l LAYOUT_FILE, --layout=LAYOUT_FILE
                           the LAYOUT_FILE from which to load the map layout
                           [Default: mediumClassic]
-p TYPE, --pacman=TYPE    the agent TYPE in the pacmanAgents module to use
                           [Default: KeyboardAgent]
-t, --textGraphics       Display output as text only
-q, --quietTextGraphics  Generate minimal output and no graphics
-g TYPE, --ghosts=TYPE   the ghost agent TYPE in the ghostAgent's module to use
                           [Default: RandomGhost]
-k NUMGHOSTS, --numghosts=NUMGHOSTS
                           The maximum number of ghosts to use [We have only 1
                           ghost, which is the default]
-z ZOOM, --zoom=ZOOM      Zoom the size of the graphics window [Default: 1.0]
-f, --fixRandomSeed       Fixes the random seed to always play the same game in
                           Part 2
-r, --recordActions       Writes game histories to a file (named by the time
                           they were played)
--replay=GAMETOREPLAY    A recorded game file (pickle) to replay
-a AGENTARGS, --agentArgs=AGENTARGS
                           Comma-separated values sent to the agent, e.g.,
                           "opt1=val1,opt2,opt3=val3"
-x NUMTRAINING, --numTraining=NUMTRAINING
                           How many episodes are training (suppresses output)
                           [Default: 0] You will need this for Assignment 3
--frameTime=FRAMETIME    Time to delay between frames; <0 means keyboard
                           [Default: 0.1]
-c, --catchExceptions     Turns off exception handling and timeouts during games
--timeout=TIMEOUT         Maximum length of time an agent can spend computing in
                           a single game [Default: 30]
-o, --outfile             output file name
```

10. Reporting Bugs

If you happen to stumble upon a bug in our code, please do the following:

1. Check to see if someone else has posted the issue already.
2. Make a post on Ed and include all the information to reproduce the issue.

11. Additional resources

If you find additional resources online, e.g., source code or algorithms,

- You need to acknowledge the source of this material, and discuss the approach in your AI declaration, as well as your extensions/modifications.
- You will need to be knowledgeable about the approach to do well in the Q&A component of the assignment.

If you use generative AI tools, e.g., ChatGPT, you must acknowledge the tools and their use in your AI declaration, and supply the prompts you employed.